

11. Interview Questions & Answers

These are the most commonly asked questions in placement interviews about Django projects:

Q1. What is Django and why did you use it for this project?

Django is a high-level Python web framework that follows the MVT (Model-View-Template) architecture. I chose Django because it comes with a built-in ORM (no raw SQL needed), authentication system, admin panel, and form handling — which saved significant development time. For a cab booking system with complex database relationships, Django's ORM made it easy to define models and query data.

Q2. Explain the MVT architecture used in this project.

Model: Defined in `cabs/models.py` — contains `Booking`, `Driver`, and `ContactMessage` classes. These map directly to database tables. View: Defined in `views.py` — contains business logic. For example, `book_cab()` calculates fare, saves booking, and redirects. Template: HTML files in `templates/` folder. Django's template engine renders dynamic data using `{{ }}` and `{% %}` tags.

Q3. How does the real-time tracking work technically?

It works using a polling mechanism: (1) Driver's phone browser calls `navigator.geolocation.getCurrentPosition()` every 10 seconds. (2) Coordinates are POSTed to `/api/driver/location/update/` which saves lat/lng to the `Driver` model in the database. (3) On the customer's track page, JavaScript calls `/api/cab/location/<booking_id>/` every 10 seconds. (4) Google Maps marker's position is updated with the new coordinates. This creates the illusion of real-time movement.

Q4. What is a ForeignKey? How did you use it here?

`ForeignKey` is a database relationship where one table references another. In this project, `Booking` has a `ForeignKey` to `Driver` — meaning each booking can have one driver assigned, but one driver can have many bookings. When Django creates the database, it adds a `driver_id` column to the `bookings` table that references the `drivers` table. I used `on_delete=models.SET_NULL` so that if a driver is deleted, their old bookings don't get deleted — just the driver reference becomes null.

Q5. What is CSRF and how does Django handle it?

CSRF (Cross-Site Request Forgery) is a security attack where a malicious website tricks a logged-in user into submitting a form to another website. Django protects against this with the `{% csrf_token %}` template tag, which adds a hidden field with a unique token to every form. The server verifies this token on form submission. I also used `@csrf_exempt` decorator on the driver location update API since it receives POST requests from mobile browsers without the token.

Q6. How did you implement the 10-minute tracking window?

In the `track_cab()` view, after finding the booking, I combine `pickup_date` and `pickup_time` into a single datetime object using `datetime.combine()`. I then use `timezone.make_aware()` to make it timezone-aware. By comparing this with `timezone.now()`, I get the difference in minutes. If `difference <= 10` minutes OR status is `on_trip`, tracking is allowed. Otherwise, I pass `minutes_until_tracking` to the template which shows a countdown message.

Q7. What is the difference between GET and POST requests in this project?

GET requests are used for: fetching pages (homepage, routes page), API calls (fare calculator, location fetch). POST requests are used for: form submissions (booking form, contact form, login), status updates (accept booking, complete trip), driver GPS push. In Django views, I check `request.method == "POST"` to handle form data, and fall through to GET handling otherwise.

Q8. How does the live alert system work without WebSockets?

Instead of WebSockets (which require additional server setup), I used polling — JavaScript calls the `/portal/api/pending/` endpoint every 10 seconds using `setInterval()`. The endpoint returns a JSON list of all pending bookings. If the count has increased since last poll, a sound plays and a browser notification fires. This approach is simpler to implement and works well for a low-traffic system. WebSockets would be more efficient at scale but require Django Channels or a separate WebSocket server.

Q9. How is the commission/revenue model implemented?

Each Booking model has a `commission_rate` field (default 15%). Two Python methods calculate the split: `admin_commission()` returns `fare × 15%` and `driver_earning()` returns `fare × 85%`. In the Revenue dashboard view, I use Django's `aggregate(Sum())` function to total up all completed booking fares, then call `admin_commission()` on each to sum up total commission. This is done for today, this month, and all time.

Q10. How did you handle authentication for two different user types?

Django's built-in User model has `is_staff` and `is_superuser` flags. Admins are superusers. Drivers are regular users but with a Driver object linked via `OneToOneField`. In the portal login view, after `authenticate()` succeeds, I check: if `user.is_staff` → admin access. If `hasattr(user, "driver")` → driver access. If neither → access denied. Throughout the portal views, `@login_required(login_url="portal_login")` ensures unauthenticated users are redirected to the portal login page, not Django admin.

Q11. What does `manage.py migrate` do?

migrate reads all migration files in each app's migrations/ folder and applies them to the database. First, makemigrations detects changes in models.py and creates migration files (Python scripts describing the changes). Then migrate executes these scripts, creating or altering actual database tables. In this project, the initial migration created 3 tables: cabs_driver, cabs_booking, and cabs_contactmessage.

Q12. What improvements would you make if given more time?

Payment Integration: Add Razorpay gateway for online payments. SMS Notifications: Use Twilio or MSG91 to SMS booking confirmation and OTP to customers. WebSocket Tracking: Replace polling with Django Channels for true real-time GPS streaming. PostgreSQL: Switch from SQLite to PostgreSQL for production scalability. Mobile App: Build a Flutter driver app instead of browser-based GPS sharing. Rating System: Allow customers to rate drivers after trip completion.